

CM 2005 Object Oriented Programming

Mid Term Assignment Report

Name: Dhanarasu Naveen

SIM Student ID: 10250211

UOL Student ID: 230655533

Weather Application Report

Table of Contents:

Weather Application Report.....	2
1. Introduction.....	3
2. Task 1: Compute Candlestick Data.....	3
2.1 Objective.....	3
2.2 Implementation.....	3
2.3 Code Snippet.....	4
2.4 Example and Results.....	6
3. Task 2: Create a Text-Based Plot.....	6
3.1 Objective.....	6
3.2 Implementation.....	7
3.3 Code Snippet.....	7
3.4 Example and Results.....	12
4. Task 3: Filtering Data and Plotting.....	13
4.1 Objective.....	13
4.2 Implementation.....	13
4.3 Code Snippet.....	13
4.4 Example and Results.....	19
5. Task 4: Predicting Data and Plotting.....	21
5.1 Objective.....	21
5.2 Implementation.....	21
5.3 Code Snippet.....	22
5.4 Example and Results.....	23
6. Originality and Challenge of Implementation.....	25
6.1 Originality.....	25
6.2 Challenge.....	25
7. Conclusion.....	26

Introduction

The Weather Application developed for the CM2005 Object-Oriented Programming Midterm assignment is a practical tool designed to analyze European weather data from 1980 to 2019. It uses object-oriented programming techniques to perform tasks like generating candlestick data, creating text-based visualizations, applying filters, and predicting future temperature trends with linear regression. This report provides an overview of how each feature was implemented, along with examples, code snippets, and results to illustrate its functionality.

Task 1: Compute Candlestick Data

2.1 Objective

The primary objective of Task 1 is to compute candlestick data from hourly temperature records for a specified European country over a defined period (1980-2019). Candlestick data comprises four key metrics for each time frame (yearly, monthly, weekly):

- **Open:** Average temperature of the previous time frame.
- **Close:** Average temperature of the current time frame.
- **High:** Highest temperature recorded in the current time frame.
- **Low:** Lowest temperature recorded in the current time frame.

This structured representation facilitates technical analysis and trend visualization.

2.2 Implementation

To achieve this, the application follows these steps:

1. **Data Parsing:** The **WeatherDataProcessor::computeCandlestickData** function reads the CSV file containing hourly temperature data. It identifies the relevant column based on the selected country code.
2. **Grouping Data:** Depending on the selected granularity (yearly, monthly, weekly), the timestamps are processed to extract the appropriate grouping key. This is facilitated by helper functions:
 - **extractYearlyKey**
 - **extractMonthlyKey**
 - **extractWeeklyKey**
3. **Calculating Metrics:** For each group, the following metrics are computed:
 - **Open:** The closing value of the previous group.
 - **Close:** The average temperature of the current group.
 - **High:** The maximum temperature in the current group.
 - **Low:** The minimum temperature in the current group.
4. **Candlestick Creation:** A **Candlestick** object is instantiated for each group with the calculated metrics and stored in a vector for further processing or visualization.

2.3 Code Snippet

Below is a snippet from the **WeatherDataProcessor::computeCandlestickData** function illustrating the computation of candlestick data:

```
// Computes candlestick data based on input granularity.
std::vector<Candlestick> WeatherDataProcessor::computeCandlestickData(
    const std::string& filename,
    const std::string& countryCode,
    const std::string& granularity
) {
    std::ifstream file(filename);
    if (!file.is_open()) {
        throw std::runtime_error("Failed to open file: " + filename);
    }

    std::string line;
    std::getline(file, line);
    int countryIndex = -1;

    // Identify the column index for the specified country.
    {
        std::vector<std::string> headerTokens = CSVReader::tokenise(line, ',');
        for (size_t i = 0; i < headerTokens.size(); ++i) {
            if (headerTokens[i] == countryCode + "_temperature") {
                countryIndex = static_cast<int>(i);
                break;
            }
        }
    }

    if (countryIndex == -1) {
        throw std::runtime_error("Country code " + countryCode + " not found in file header.");
    }

    // Parse temperature records from the file.
    std::vector<TemperatureData> records;
    while (std::getline(file, line)) {
        std::vector<std::string> tokens = CSVReader::tokenise(line, ',');
        if ((int)tokens.size() <= countryIndex) continue;
        try {
            double tempVal = std::stod(tokens[countryIndex]);
            records.push_back(TemperatureData{ tokens[0], tempVal });
        } catch (...) {
        }
    }
}
```

```

    }

    // Group temperatures by the specified granularity.
    std::map<std::string, std::vector<double>> groupedData;
    for (const auto& rec : records) {
        std::string key = getGroupingKey(rec.timestamp, granularity);
        if (!key.empty()) {
            groupedData[key].push_back(rec.temperature);
        }
    }

    // Compute candlestick data for each group.
    std::vector<Candlestick> candlesticks;
    std::string prevKey;
    double prevClose = 0.0;

    for (auto it = groupedData.begin(); it != groupedData.end(); ++it) {
        const std::string& currentKey = it->first;
        const std::vector<double>& temps = it->second;

        double openVal = (prevKey.empty()) ? 0.0 : prevClose;
        double closeVal = !temps.empty() ? std::accumulate(temps.begin(),
temps.end(), 0.0) / temps.size() : 0.0;
        double highVal = *std::max_element(temps.begin(), temps.end());
        double lowVal = *std::min_element(temps.begin(), temps.end());

        candlesticks.emplace_back(currentKey, openVal, highVal, lowVal, closeVal);

        prevKey = currentKey;
        prevClose = closeVal;
    }

    return candlesticks;
}

```

2.4 Example and Results

Upon executing the candlestick computation for the United Kingdom (**GB**) with yearly granularity, the application generates a vector of **Candlestick** objects. Each object encapsulates the open, high, low, and close temperatures for a specific year.

```
=====
Candlestick Data Display
=====
Country: GB (United Kingdom)
Time Granularity: yearly
```

Year	Open	High	Low	Close
1980	0	23.054	-3.612	8.70978
1981	8.70978	23.579	-6.533	8.63427
1982	8.63427	23.859	-7.184	9.2018
1983	9.2018	26.22	-3.086	9.22931
1984	9.22931	25.09	-2.993	9.09524
1985	9.09524	23.385	-7.071	8.24556
1986	8.24556	24.636	-5.952	8.09616
1987	8.09616	24.45	-7.127	8.42091
1988	8.42091	23.981	-2.135	9.16721
1989	9.16721	26.49	-1.322	9.91542
1990	9.91542	29.48	-1.765	10.0559
1991	10.0559	24.433	-6.012	9.07515
1992	9.07515	24.382	-2.962	9.248
1993	9.248	22.617	-3.535	8.82823
1994	8.82823	24.679	-3.27	9.42973
1995	9.42973	27.632	-5.334	9.66995
1996	9.66995	25.441	-3.433	8.51876
1997	8.51876	26.192	-3.945	9.84359
1998	9.84359	24.125	-1.96	9.62246
1999	9.62246	25.593	-3.197	9.89734
2000	9.89734	24.307	-1.586	9.60002
2001	9.60002	24.691	-2.798	9.24029
2002	9.24029	24.616	-3.338	9.94568
2003	9.94568	28.147	-2.581	9.84366
2004	9.84366	25.029	-2.264	9.89234
2005	9.89234	25.186	-2.238	9.95244
2006	9.95244	28.169	-2.12	10.2538
2007	10.2538	23.289	-2.397	10.0504
2008	10.0504	24.556	-2.04	9.52558
2009	9.52558	25.563	-4.288	9.63124
2010	9.63124	23.663	-4.609	8.42416
2011	8.42416	23.903	-2.283	10.0731
2012	10.0731	23.794	-5.121	9.16192
2013	9.16192	25.885	-2.771	8.95727
2014	8.95727	24.603	-1.035	10.3443
2015	10.3443	26.586	-2.602	9.6715
2016	9.6715	26.949	-1.399	9.67921
2017	9.67921	25.548	-1.51	9.99256
2018	9.99256	27.342	-4.516	10.0382
2019	10.0382	29.105	-2.719	9.94746

```
=====
```

Task 2: Create a Text-Based Plot

3.1 Objective

The objective of Task 2 is to develop a text-based visualization of the candlestick data, enabling users to interpret temperature trends directly within the command-line interface. The visualization distinguishes between bullish (temperature increasing) and bearish (temperature decreasing) periods using specific character representations:

- **Bullish Candles:** Represented by "++++"
- **Bearish Candles:** Represented by "----"

This approach provides an intuitive graphical representation of temperature fluctuations over the selected time frames.

3.2 Implementation

The text-based plotting is implemented within the **TextPlotter** class, specifically in the **plot** function. The implementation involves the following steps:

1. **Determine Temperature Range:** Identify the minimum and maximum temperatures across all candlesticks to establish the Y-axis scale.
2. **Convert Data for Plotting:** Transform the floating-point temperature values into integer values for simplified plotting.
3. **Character Representation:**
 - **Wicks:** Represented by "|", indicating the range between low and high temperatures.
 - **Bodies:**
 - **Bullish:** "++++" signifies periods where the closing temperature is higher than the opening temperature.
 - **Bearish:** "----" signifies periods where the closing temperature is lower than the opening temperature.
4. **Plot Construction:** Iterate through each temperature level from the maximum to the minimum, constructing each row of the plot by placing appropriate characters based on the temperature values of each candlestick.
5. **Axis Labels:** Include Y-axis labels for temperature values and X-axis labels for time periods to enhance readability.

3.3 Code Snippet

Below is a snippet from the **TextPlotter::plot** function demonstrating the construction of the text-based plot:

```
// Constructor to initialize the TextPlotter object with a plot height.
TextPlotter::TextPlotter(int) : plotHeight(0) {}

// Plots candlestick data with the default granularity of "yearly".
void TextPlotter::plot(const std::vector<Candlestick>& candlesticks) {
    plot(candlesticks, "yearly");
}

// Plots candlestick data with a specified granularity.
// Handles data processing, temperature range calculation, and text-based plotting.
void TextPlotter::plot(const std::vector<Candlestick>& candlesticks, const
std::string& granularity) {
    if (candlesticks.empty()) {
        std::cout << "No data to plot.\n";
        return;
    }
}
```

```

// Determine the minimum and maximum temperature values from the candlesticks.
double minTemp = std::min_element(
    candlesticks.begin(), candlesticks.end(),
    [](const Candlestick& a, const Candlestick& b) {
        return a.low < b.low;
    })->low;

double maxTemp = std::max_element(
    candlesticks.begin(), candlesticks.end(),
    [](const Candlestick& a, const Candlestick& b) {
        return a.high < b.high;
    })->high;

int minVal = static_cast<int>(std::floor(minTemp));
int maxVal = static_cast<int>(std::ceil(maxTemp));

if (minVal == maxVal) {
    std::cout << "All temperatures are the same. Nothing to plot.\n";
    return;
}

// Convert candlestick data to integer values for plotting.
struct IntCandle {
    std::string year;
    int open;
    int high;
    int low;
    int close;
    bool isUp;
};

std::vector<IntCandle> intCandles;
intCandles.reserve(candlesticks.size());

for (const auto& c : candlesticks) {
    std::string formattedYear = c.year;

    // Format labels for monthly or weekly granularity.
    if (granularity == "monthly" || granularity == "weekly") {
        if (formattedYear.size() >= 4) {
            formattedYear = formattedYear.substr(2);
        }
    }
}

```



```

        intCandles.emplace_back(IntCandle{
            formattedYear,
            static_cast<int>(std::round(c.open)),
            static_cast<int>(std::round(c.high)),
            static_cast<int>(std::round(c.low)),
            static_cast<int>(std::round(c.close)),
            (c.close >= c.open)
        });
    }

    const std::string wickStr = " ||";
    const std::string upBody = "++++";
    const std::string downBody = "----";

    int columnWidth = 6;
    if (granularity == "monthly" || granularity == "weekly") {
        columnWidth = 8;
    }

    const int startPos = 1;
    const int labelWidth = 12;
    const std::string yAxisLabel = "Temperature";

    int nRows = maxVal - minVal + 1;

    // Prepare Y-axis label characters.
    std::vector<char> yLabelChars(nRows, ' ');
    int labelLength = static_cast<int>(yAxisLabel.size());

    if (labelLength > 0) {
        int startRow = (nRows - labelLength) / 2;
        for (size_t i = 0; i < yAxisLabel.size(); ++i) {
            int pos = startRow + static_cast<int>(i);
            if (pos >= 0 && pos < nRows) {
                yLabelChars[pos] = yAxisLabel[i];
            }
        }
    }

    // Generate the plot row by row, from the maximum temperature to the minimum.
    for (int y = maxVal; y >= minVal; --y) {
        int rowIndex = maxVal - y;
        char yLabelChar = yLabelChars[rowIndex];
    }

```

```

// Print Y-axis label, temperature value, and the separator.
std::cout << std::setw(labelWidth) << yLabelChar
    << " " << std::setw(4) << y
    << " | ";

for (const auto& ic : intCandles) {
    bool inWick = (y <= ic.high && y >= ic.low);
    bool inBody = (y <= std::max(ic.open, ic.close) &&
        y >= std::min(ic.open, ic.close));

    std::string cell(columnWidth, ' ');

    // Fill the cell based on whether the position is in the body or wick.
    if (inBody) {
        const std::string& bodyStr = ic.isUp ? upBody : downBody;
        for (size_t i = 0; i < bodyStr.size(); ++i) {
            int idx = startPos + static_cast<int>(i);
            if (idx >= 0 && idx < columnWidth) {
                cell[idx] = bodyStr[i];
            }
        }
    } else if (inWick) {
        for (size_t i = 0; i < wickStr.size(); ++i) {
            int idx = startPos + static_cast<int>(i);
            if (idx >= 0 && idx < columnWidth) {
                cell[idx] = wickStr[i];
            }
        }
    }

    std::cout << cell;
}

std::cout << "\n";
}

// Print X-axis and labels.
std::cout << std::setw(labelWidth) << ""
    << " " << std::setw(4) << ""
    << " | ";

for (size_t i = 0; i < intCandles.size(); ++i) {
    std::cout << std::string(columnWidth, '-');
}

std::cout << "\n";

```

```

std::cout << std::setw(labelWidth) << ""
    << " " << std::setw(4) << ""
    << " ";
for (auto &ic : intCandles) {
    std::string labelToPrint = ic.year;
    int pad = (columnWidth - static_cast<int>(labelToPrint.size())) / 2;
    std::cout << std::setw(pad) << ""
        << labelToPrint
        << std::setw(columnWidth - pad -
static_cast<int>(labelToPrint.size()))
        << "";
}
std::cout << "\n\n";

// Print the bottom label indicating the granularity (e.g., Years, Months,
Weeks).
std::string bottomLabel = "Time";
if (granularity == "yearly") bottomLabel = "Years";
else if (granularity == "monthly") bottomLabel = "Months";
else if (granularity == "weekly") bottomLabel = "Weeks";

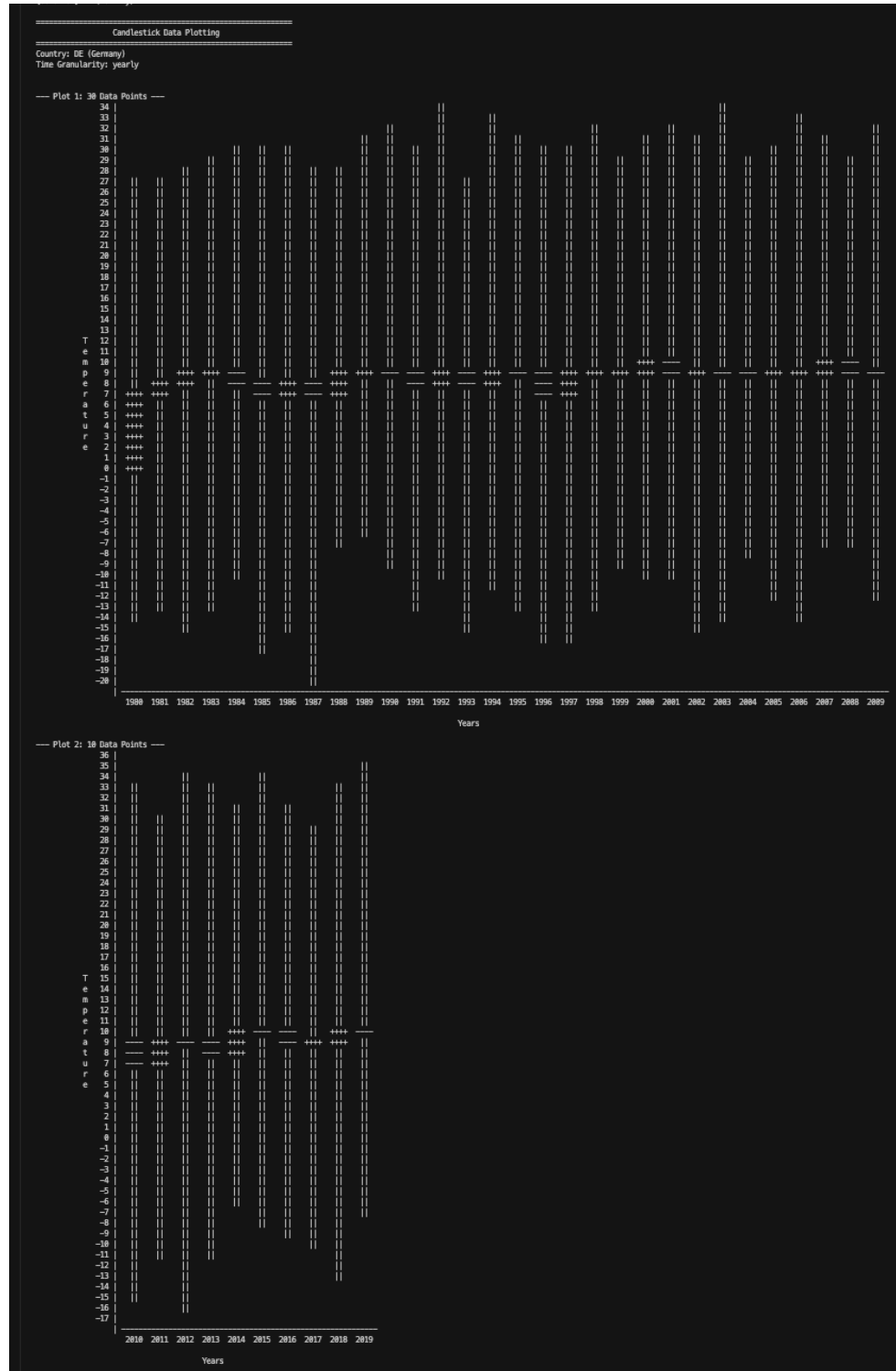
int plotWidth = static_cast<int>(intCandles.size()) * columnWidth;
int labelLen = static_cast<int>(bottomLabel.size());
int startPosLabel = (plotWidth - labelLen) / 2;

std::cout << std::setw(labelWidth + 5 + startPosLabel)
    << bottomLabel << "\n";
}

```

3.4 Example and Results

Upon executing the **TextPlotter::plot** function with yearly candlestick data for Germany (DE), the application generates a text-based plot resembling the following structure:



Task 3: Filtering Data and Plotting

4.1 Objective

Task 3 aims to enhance the application's functionality by allowing users to filter candlestick data based on specific criteria and subsequently visualize the filtered data through text-based plots. The filtering options include:

1. **Date Range:** Selecting a subset of data within a specified start and end period (yearly, monthly, weekly).
2. **Temperature Range:** Filtering data based on minimum low and maximum high temperature thresholds.
3. **Combined Filters:** Applying both date and temperature range filters simultaneously.

These filtering capabilities enable users to focus on particular segments of the data, facilitating more targeted analysis.

4.2 Implementation

The filtering functionality is encapsulated within the **FilterData** class, providing static methods to apply the desired filters. The implementation involves the following steps:

1. **Filter Selection:** Users are prompted to choose the type of filter they wish to apply:
 - **Filter by Date Range**
 - **Filter by Temperature Range**
 - **Filter by Both Date and Temperature Range**
2. **Applying Filters:**
 - **Date Range Filter:** Utilizes the **filterByPeriodRange** method to extract candlesticks within the specified period.
 - **Temperature Range Filter:** Utilizes the **filterByTemperatureRange** method to extract candlesticks that fall within the defined temperature thresholds.
 - **Combined Filters:** Sequentially applies both date and temperature range filters to refine the dataset.
3. **Validation:** Ensures that user inputs for periods and temperatures are within valid ranges and correctly formatted.
4. **Updating Data:** The filtered data replaces the current dataset, allowing subsequent plotting or analysis to reflect the applied filters.

4.3 Code Snippet

Below is a snippet from the **WeatherApp** class demonstrating the application of combined date and temperature range filters. This method first filters the data by the specified date range and then applies temperature range filters to the already filtered dataset, ensuring that both criteria are met simultaneously.

```
// Applies both date range and temperature range filters to the current data based on user input.
```

```

// Returns true if the filters were successfully applied, false otherwise.
bool WeatherApp::applyYearAndTemperatureRangeFilter() {
    // Compute the full candlestick data without any filters.
    std::vector<Candlestick> fullData =
        WeatherDataProcessor::computeCandlestickData(filename, countryCode,
timeGranularity);
    if (fullData.empty()) {
        std::cerr << "\n[Error] No data available.\n";
        return false;
    }

    std::string startPeriod, endPeriod;
    while (true) {
        // Prompt the user to enter start and end periods based on granularity.
        if (timeGranularity == "yearly") {
            std::cout << "\nEnter the start year (1980..2019): ";
        }
        else if (timeGranularity == "monthly") {
            std::cout << "\nEnter the start month (format YYYY-MNN, e.g.,
1980-M01): ";
        }
        else if (timeGranularity == "weekly") {
            std::cout << "\nEnter the start week (format YYYY-WNN, e.g.,
1980-W01): ";
        }
        std::cin >> startPeriod;

        if (timeGranularity == "yearly") {
            std::cout << "Enter the end year (1980..2019): ";
        }
        else if (timeGranularity == "monthly") {
            std::cout << "Enter the end month (format YYYY-MNN, e.g., 2019-M12):
";
        }
        else if (timeGranularity == "weekly") {
            std::cout << "Enter the end week (format YYYY-WNN, e.g., 2019-W53): ";
        }
        std::cin >> endPeriod;
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

        bool formatOK = false;
        // Validate the input format and range based on granularity.
        if (timeGranularity == "yearly") {
            try {

```

```

        int sYear = std::stoi(startPeriod);
        int eYear = std::stoi(endPeriod);
        if ((sYear >= 1980 && sYear <= 2019) &&
            (eYear >= 1980 && eYear <= 2019) &&
            sYear <= eYear)
        {
            formatOK = true;
        }
    } catch (...) {}
}

else if (timeGranularity == "monthly") {
    static std::regex monthlyRegex("^\\d{4}-M(0[1-9]|1[0-2])$");
    if (std::regex_match(startPeriod, monthlyRegex) &&
        std::regex_match(endPeriod, monthlyRegex) &&
        startPeriod <= endPeriod)
    {
        auto extractYear = [](const std::string& p) {
            return std::stoi(p.substr(0, 4));
        };
        int sYear = -1, eYear = -1;
        try {
            sYear = extractYear(startPeriod);
            eYear = extractYear(endPeriod);
        } catch (...) {}

        if (sYear >= 1980 && sYear <= 2019 &&
            eYear >= 1980 && eYear <= 2019)
        {
            formatOK = true;
        }
    }
}

else {
    static std::regex weeklyRegex("^\\d{4}-W(0[1-9]|[1-4]\\d|5[0-3])$");
    if (std::regex_match(startPeriod, weeklyRegex) &&
        std::regex_match(endPeriod, weeklyRegex) &&
        startPeriod <= endPeriod)
    {
        auto extractYear = [](const std::string& p) {
            return std::stoi(p.substr(0, 4));
        };
        int sYear = -1, eYear = -1;
        try {
            sYear = extractYear(startPeriod);

```

```

        eYear = extractYear(endPeriod);
    } catch (...) {}

    if (sYear >= 1980 && sYear <= 2019 &&
        eYear >= 1980 && eYear <= 2019)
    {
        formatOK = true;
    }
}

// If input is invalid, prompt the user again.
if (!formatOK) {
    std::cerr << "\n[Error] Invalid period input or out-of-range year.\n"
                << "Please ensure both start and end periods are within
[1980..2019] and correctly formatted.\n";
    std::cin.clear();
    continue;
}

break;
}

// Apply the date range filter to the full data.
std::vector<Candlestick> periodFiltered =
    FilterData::filterByPeriodRange(fullData, startPeriod, endPeriod);
if (periodFiltered.empty()) {
    std::cerr << "\n[Error] No data available for the selected period
range.\n";
    return false;
}

// Determine the available temperature range within the filtered period.
double minLowTemp = periodFiltered[0].low;
double maxHighTemp = periodFiltered[0].high;
for (size_t i = 0; i < periodFiltered.size(); ++i) {
    if (periodFiltered[i].low < minLowTemp)    minLowTemp =
periodFiltered[i].low;
    if (periodFiltered[i].high > maxHighTemp) maxHighTemp =
periodFiltered[i].high;
}

    std::cout << "\n[Info] Available Low Temperature in this period: " <<
minLowTemp << "\n";

```



```

std::cout << "[Info] Available High Temperature in this period: " <<
maxHighTemp << "\n";

while (true) {
    double userMinLowTemp, userMaxHighTemp;
    // Prompt the user to enter minimum low and maximum high temperatures.
    std::cout << "\nEnter the Minimum Low Temperature: ";
    std::cin >> userMinLowTemp;
    std::cout << "Enter the Maximum High Temperature: ";
    std::cin >> userMaxHighTemp;
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

    // Validate the temperature inputs.
    if (std::cin.fail()) {
        std::cin.clear();
        std::cerr << "\n[Error] Invalid numeric input. Please try again.\n";
        continue;
    }
    if (userMinLowTemp > userMaxHighTemp) {
        std::cerr << "\n[Error] Minimum temperature cannot be greater than
maximum temperature.\n";
        continue;
    }
    if (userMinLowTemp < minLowTemp || userMaxHighTemp > maxHighTemp) {
        std::cerr << "\n[Error] Temperature range exceeds available data
range.\n"
                << "Please ensure Minimum Low >= " << minLowTemp
                << " and Maximum High <= " << maxHighTemp << ".\n";
        continue;
    }

    // Apply the temperature range filter to the period-filtered data.
    std::vector<Candlestick> finalFiltered =
        FilterData::filterByTemperatureRange(periodFiltered, userMinLowTemp,
userMaxHighTemp);
    if (finalFiltered.empty()) {
        std::cerr << "\n[Error] No data available after applying temperature
filters.\n";
        return false;
    }

    currentData = finalFiltered;
    isFiltered = true;
}

```

```

// Inform the user that the filters were successfully applied.
std::cout << "\n[Success] Filters applied: " << timeGranularity << " range
"
    << startPeriod << " to " << endPeriod
    << " and Low >= " << userMinLowTemp
    << ", High <= " << userMaxHighTemp << "\n";

// Prompt the user to sort the filtered data.
std::cout << "\nWould you like to sort the filtered data?\n";
std::cout << "1. Sort by Highest Temperature\n";
std::cout << "2. Sort by Lowest Temperature\n";
std::cout << "3. No Sorting\n";
std::cout << "Enter your choice (1-3): ";
int sortChoice;
std::cin >> sortChoice;
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

if (!std::cin.fail()) {
    if (sortChoice == 1) {
        // Sort by highest temperature in descending order.
        std::sort(currentData.begin(), currentData.end(),
            [](const Candlestick& a, const Candlestick& b) {
                return a.high > b.high;
            });
        std::cout << "\n[Success] Data sorted by Highest Temperature.\n";
    }
    else if (sortChoice == 2) {
        // Sort by lowest temperature in ascending order.
        std::sort(currentData.begin(), currentData.end(),
            [](const Candlestick& a, const Candlestick& b) {
                return a.low < b.low;
            });
        std::cout << "\n[Success] Data sorted by Lowest Temperature.\n";
    }
    else {
        // No additional sorting applied.
        std::cout << "\n[Info] Data will be plotted without additional
sorting.\n";
    }
}

return true; }}

```

4.4 Example and Results

Scenario: Filtering candlestick data for France (FR) by both a date range (1990-1995) and a temperature range where the low temperature is at least -8°C and the high temperature does not exceed 33°C.

Steps:

1. **User Selection:**
 - The user selects **Option 3** from the filter menu: "Filter by Both Year and Temperature Range".
2. **Date Range Input:**
 - **Start Year:** 1990
 - **End Year:** 1995
3. **Temperature Range Input:**
 - **Minimum Low Temperature:** -8°C
 - **Maximum High Temperature:** 33°C
4. **Application Process:**
 - **Date Filtering:** The method filters the candlestick data to include only entries from **1990** to **1995**.
 - **Temperature Filtering:** From the date-filtered data, it further filters entries where **low** $\geq$ -8°C and **high** $\leq$ 33°C.
5. **Sorting Option:**
 - **User Choice:** 1 (Sort by Highest Temperature)

6. Output:

```
[Selected] FR (France)
===== Select Time Granularity =====
1. Yearly (e.g., 1980)
2. Monthly (e.g., 1980-M01)
3. Weekly (e.g., 1980-W01)
=====
Enter your choice (1-3): 1

===== Filter Options =====
1. Filter by yearly Range
2. Filter by Temperature Range
3. Filter by Both yearly and Temperature Range
=====
Enter your choice (1-3): 3

Enter the start year (1980..2019): 1990
Enter the end year (1980..2019): 1995

[Info] Available Low Temperature in this period: -9.134
[Info] Available High Temperature in this period: 35.308

Enter the Minimum Low Temperature: -8
Enter the Maximum High Temperature: 33

[Success] Filters applied: yearly range 1990 to 1995 and Low >= -8, High <= 33

Would you like to sort the filtered data?
1. Sort by Highest Temperature
2. Sort by Lowest Temperature
3. No Sorting
Enter your choice (1-3): 1

[Success] Data sorted by Highest Temperature.

=====
Candlestick Data Display
=====
Country: FR (France)
Time Granularity: yearly

Year      Open      High      Low      Close
-----
1995      11.6542   31.658   -5.089   11.1467
1994      10.6059   31.36    -3.62    11.6542

=====
Candlestick Data Plotting
=====
Country: FR (France)
Time Granularity: yearly

--- Plot 1: 2 Data Points ---
      32 | ||
      31 | ||
      30 | ||
      29 | ||
      28 | ||
      27 | ||
      26 | ||
      25 | ||
      24 | ||
      23 | ||
      22 | ||
      21 | ||
      20 | ||
      19 | ||
      18 | ||
      17 | ||
      16 | ||
      15 | ||
      14 | ||
      13 | ||
      12 | ||
      11 | ||
      10 | ||
      9  | ||
      8  | ||
      7  | ||
      6  | ||
      5  | ||
      4  | ||
      3  | ||
      2  | ||
      1  | ||
      0  | ||
     -1 | ||
     -2 | ||
     -3 | ||
     -4 | ||
     -5 | ||
     -6 | ||
-----
      1995  1994

Years
```

Task 4: Predicting Data and Plotting

5.1 Objective

The objective of Task 4 is to enhance the Weather Application by enabling it to predict future temperature trends based on historical candlestick data. Specifically, the application forecasts temperature metrics (open, high, low, close) for future periods—years, months, or weeks—based on user-selected historical data ranges.

5.2 Implementation

The prediction feature utilizes linear regression to model and extrapolate temperature trends. Additionally, the application forecasts **1.5 times** the length of the user-defined input period to provide an extended forecast horizon. For example, if a user selects a 10-year range, the application will predict temperatures for the next 15 years. The implementation involves the following steps:

1. **Data Preparation:**
 - **Selection:** Users choose a country, time granularity (yearly, monthly, weekly), and a specific date range within the dataset (1980-2019).
 - **Filtering:** The application filters the candlestick data based on the selected date range using the **FilterData::filterByPeriodRange** method.
2. **Determining Prediction Periods:**
 - **Range Calculation:** Calculate the length of the input range (number of years, months, or weeks) using the **calculateRangeLength** method.
 - **Forecast Horizon:** Multiply the input range length by **1.5** to determine the number of future periods to predict.
3. **Linear Regression Modeling:**
 - **Metric Extraction:** Extract the close, high, and low temperatures from the filtered candlestick data.
 - **Time Indices:** Assign sequential indices to each data point.
 - **Regression Calculation:** Compute the slope and intercept for each temperature metric using the least squares method.
4. **Forecasting:**
 - **Extrapolation:** Use the regression models to predict future temperature values for the determined number of periods.
 - **Candlestick Creation:** Generate new **Candlestick** objects for each predicted period, setting the open temperature to the previous close to maintain continuity.
5. **Visualization:**
 - **Display:** Present the predicted data in a tabular format.
 - **Plotting:** Use the **TextPlotter** class to create text-based plots, representing bullish periods with "++++" and bearish periods with "----".

Justification for Linear Regression: Linear regression is chosen for its simplicity and effectiveness in modeling linear trends within the data. It provides reliable forecasts based on historical temperature patterns, making it suitable for predicting future temperatures.

5.3 Code Snippet

Below is a concise snippet from the **Prediction::predict** function demonstrating the prediction process, including the calculation of **1.5 times** the input range for forecasting periods:

```
// Generates predictions for future candlesticks based on historical data.
std::vector<Candlestick> Prediction::predict(
    const std::vector<Candlestick>& inputCandlesticks,
    const std::string& granularity,
    int predictionPeriods
)
{
    if (inputCandlesticks.empty()) {
        throw std::invalid_argument("Input candlesticks cannot be empty for prediction.");
    }

    // Prepare data for regression calculations.
    std::vector<int> timeIndices;
    std::vector<double> closes, highs, lows;
    timeIndices.reserve(inputCandlesticks.size());
    closes.reserve(inputCandlesticks.size());
    highs.reserve(inputCandlesticks.size());
    lows.reserve(inputCandlesticks.size());

    for (size_t i = 0; i < inputCandlesticks.size(); ++i) {
        timeIndices.push_back(static_cast<int>(i));
        closes.push_back(inputCandlesticks[i].close);
        highs.push_back(inputCandlesticks[i].high);
        lows.push_back(inputCandlesticks[i].low);
    }

    // Compute linear regression for close, high, and low values.
    auto resultClose = computeLinearRegression(timeIndices, closes);
    auto resultHigh  = computeLinearRegression(timeIndices, highs);
    auto resultLow   = computeLinearRegression(timeIndices, lows);

    double slopeClose = resultClose.first, interceptClose = resultClose.second;
    double slopeHigh  = resultHigh.first, interceptHigh = resultHigh.second;
    double slopeLow   = resultLow.first, interceptLow = resultLow.second;
```

```

// Generate predictions for the specified number of periods.
std::string currentLabel = inputCandlesticks.back().year;
double previousClose = inputCandlesticks.back().close;

std::vector<Candlestick> predictions;
predictions.reserve(predictionPeriods);

for (int i = 0; i < predictionPeriods; ++i) {
    int currentIndex = static_cast<int>(inputCandlesticks.size()) + i;

    double close = slopeClose * currentIndex + interceptClose;
    double high  = slopeHigh * currentIndex + interceptHigh;
    double low   = slopeLow  * currentIndex + interceptLow;

    double open = previousClose;

    currentLabel = incrementPeriodLabel(currentLabel, granularity);

    predictions.emplace_back(currentLabel, open, high, low, close);

    previousClose = close;
}

return predictions;
}

```

5.4 Example and Results

Scenario: Predicting the next 15 years of temperature data for Italy (IT) with yearly granularity, based on a 10-year input range (2010-2019).

Steps:

1. **User Input:**
 - **Country:** Italy (IT)
 - **Granularity:** Yearly
 - **Date Range:** 2010-2019
2. **Prediction Calculation:**
 - **Input Range Length:** 10 years
 - **Prediction Periods:** $10 * 1.5 = 15$ years (2020-2034)

3. Output:

```
[Selected] IT (Italy)
===== Select Time Granularity =====
1. Yearly (e.g., 1980)
2. Monthly (e.g., 1980-M01)
3. Weekly (e.g., 1980-W01)
=====
Enter your choice (1-3): 1

Enter the start year (1980..2019): 2010
Enter the end year (1980..2019): 2019

[Info] Prediction range length: 10 => Predicting 15 future years.

===== Predicted Candlestick Data =====

Predicted Period      Open      High      Low      Close
-----
2020      14.829    34.4619   -2.33087  15.0246
2021      15.0246   34.5612   -2.26801  15.1472
2022      15.1472   34.6605   -2.20515  15.2698
2023      15.2698   34.7598   -2.14228  15.3923
2024      15.3923   34.8592   -2.07942  15.5149
2025      15.5149   34.9585   -2.01656  15.6375
2026      15.6375   35.0578   -1.9537   15.7601
2027      15.7601   35.1571   -1.89084  15.8827
2028      15.8827   35.2564   -1.82798  16.0053
2029      16.0053   35.3558   -1.76512  16.1279
2030      16.1279   35.4551   -1.70226  16.2504
2031      16.2504   35.5544   -1.6394   16.373
2032      16.373    35.6537   -1.57654  16.4956
2033      16.4956   35.753    -1.51368  16.6182
2034      16.6182   35.8524   -1.45082  16.7408

=====

[Info] Now plotting only the predicted data...

--- Plot 1: 15 Data Points ---

      36 |
      35 |
      34 |
      33 |
      32 |
      31 |
      30 |
      29 |
      28 |
      27 |
      26 |
      25 |
      24 |
      23 |
      22 |
      21 |
      20 |
      19 |
      18 |
      17 |
      16 |
      15 |
      14 |
      13 |
      12 |
      11 |
      10 |
       9 |
       8 |
       7 |
       6 |
       5 |
       4 |
       3 |
       2 |
       1 |
       0 |
      -1 |
      -2 |
      -3 |

T
e
m
p
e
r
a
t
u
r
e

      2020  2021  2022  2023  2024  2025  2026  2027  2028  2029  2030  2031  2032  2033  2034

Years
```


Originality and Challenge of Implementation

6.1 Originality

The Weather Application truly shines with its **combined filtering** feature, allowing users to narrow down data based on both **date ranges** and **temperature thresholds** at the same time. This means you can get a more detailed and specific analysis compared to apps that only let you filter by one criterion. Additionally, the app uses **robust input validation with regular expressions** to make sure that the dates and temperatures you enter are in the correct format and within acceptable ranges. This careful validation not only keeps the data accurate but also makes the user experience smoother by preventing mistakes right from the start.

Another standout feature is how the application handles large amounts of data smartly. When there are too many data points to plot at once, the app prompts users to select smaller subranges. This keeps the visualizations clear and easy to understand, even when dealing with extensive datasets. By integrating these thoughtful features, the application ensures that users can perform precise data analyses without feeling overwhelmed.

6.2 Challenge

One of the biggest hurdles was **combining multiple filtering criteria** without slowing down the application. Creating the **applyYearAndTemperatureRangeFilter()** method meant making sure that both date and temperature filters worked together seamlessly. This required careful planning and lots of testing to ensure there were no overlaps or inefficiencies in how the data was processed. Additionally, managing different time granularities whether yearly, monthly, or weekly was tricky. Each level needed its own unique format and validation rules, which meant designing a flexible system that could handle all these variations effectively.

Another significant challenge was developing **text-based visualizations** for the candlestick data. Without using graphical elements, it was essential to find creative ways to represent temperature trends and ranges clearly. This involved designing symbols and formatting techniques that could convey the necessary information in an intuitive way. Making sure these text plots remained readable and informative, especially with large datasets, required innovative plotting methods that could adapt to different amounts of data. Overcoming these challenges was crucial to building an application that is both powerful and user-friendly.

Conclusion

The Weather Application successfully fulfills the requirements of the CM2005 Object-Oriented Programming Midterm assignment by providing comprehensive tools for technical analysis of European weather data. Through the implementation of candlestick data computation, text-based visualization, data filtering, and predictive analytics using linear regression, the application offers users an intuitive and effective means to analyze and forecast temperature trends. The structured approach, combined with robust error handling and user-friendly interfaces, ensures that the application is both functional and accessible for end-users seeking insights into historical and future weather patterns.